ACHARYA INSTITUTE OF GRADUATE STUDIES
(AFFILIATED TO BENGALURU CITY UNIVERSITY)

TRAINING PROJECT REPORT ON

EVENT MANAGEMENT SYSTEM

SUBMITTED BY

Daniel Biju Jose

Bhoomika UB

Shravya R

Submitted in partial fulfillment of the requirements for the award of the degree of

Master of Computer Applications (MCA)

Under the Guidance of

Rajesh Kumar

Department of Computer Applications

ACADEMIC YEAR 2025-2026

DECLARATION

We, the undersigned students of Bachelor of Computer Applications, hereby declare that the project report titled “EVENT MANAGEMENT SYSTEM” is our original work carried out under the guidance of Rajesh Kumar . This report has not been submitted earlier to any other university or institution for the award of any degree or diploma
.

Daniel Biju Jose
Bhoomika UB
Shravya R

Date: March 2026
Place: Bengaluru

CERTIFICATE

This is to certify that the project report titled “EVENT MANAGEMENT SYSTEM” submitted by Daniel Biju Jose, Bhoomika UB, Shravya R in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications is a record of bonafide work carried out by them under my supervision and guidance. The work has not been submitted elsewhere for any other degree or diploma.

Rajesh Kumar
Project Guide

Department of Computer Applications
Acharya Institute of Graduate Studies

Date: March 2026

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide [Faculty Name] for constant support and valuable suggestions. We thank the Head of the Department and all faculty members for providing the necessary facilities. Special thanks to our family and friends for their encouragement.

Daniel Biju Jose
Bhoomika UB
Shravya R

EVENT MANAGEMENT SYSTEM

INDEX

1. OBJECTIVE OF THE PROJECT

The primary objective of the Event Management System is to develop a robust, user-friendly console-based application that streamlines the entire lifecycle of event planning and execution. In today's fast-paced world, organizing events—whether academic fests, corporate seminars, weddings, or cultural programs—requires meticulous coordination of multiple resources, team members, responsibilities, and financial tracking. Manual methods using paper registers or scattered Excel sheets often lead to miscommunication, missed deadlines, and budget overruns.

This project aims to provide a centralized digital solution using Java and MySQL that enables authenticated users to:

- Register and securely log in.
- Create a new event with name, objective, and estimated budget.
- Add and manage resources (venue items, equipment, etc.).
- Maintain a team member database with roles.
- Assign specific tasks with deadlines and track their status (Pending → Completed).
- Record all expenses with descriptions.
- Generate consolidated reports that join responsibilities and resources with event names for better visibility.

Additional objectives include:

- Enforcing a simple authentication mechanism to prevent unauthorized access.
- Providing real-time task status updates via database operations.
- Ensuring data persistence across sessions through relational database storage.
- Offering menu-driven navigation for ease of use by non-technical users.
- Demonstrating practical application of JDBC connectivity, PreparedStatement for security against SQL injection, and ResultSet processing.

The system is developed entirely in Java 8+ with MySQL 8.0, making it platform-independent and cost-free for academic and small-scale deployments By achieving these objectives, the system significantly reduces human error, saves time, and provides an auditable trail of all event activities. The project also serves as an academic demonstration of layered architecture (Controller → DAO → Model → Database) in Java console applications.
Detailed Benefits Realized

Efficiency: Tasks that previously took hours of manual coordination can now be completed in minutes.
Accuracy: Budget and expense tracking prevents overspending.
Accountability: Every task is linked to a team member with a clear deadline.
Scalability: Although currently limited to one active event, the design can be extended to multi-event support.

2. SCOPE OF THE PROJECT

The scope is deliberately focused to deliver a fully functional minimum viable product (MVP) within the constraints of a training project:
In-Scope Features

User Management – Registration and login (role stored but not yet enforced).
Single Event Creation (global static eventId for simplicity).
Resource Management – Add and view with event linkage.
Team Member Management – Add members with roles.
Responsibility (Task) Management – Assign, view, mark complete, delete, pending tasks, and joined view.
Expense Tracking – Add and record against the event.
Reporting – Two specialized JOIN queries for responsibilities and resources with event name.

Out-of-Scope (Future Phases)
Multiple simultaneous events per user.
Graphical User Interface (Swing/JavaFX).
Role-based access control (Admin vs User).
Password hashing & salting.
Budget vs actual expense analytics.
Export to PDF/Excel.
Cloud deployment.
Mobile application integration.

Technical Scope

Console-based only (no GUI).
Hard-coded database credentials in DBConnection.java.
MySQL as bac kend (no ORM like Hibernate).
Single-user concurrent access (no multi-threading).

The scope ensures the project remains manageable while covering all core CRUD operations and relational joins, providing a solid foundation for future enhancements.

3. FEASIBILITY STUDY

3.1 Technical Feasibility

All technologies used are open-source and widely supported:
Java (JDBC API built-in).
MySQL Community Edition.
No additional paid libraries required.
The application runs on any Windows/Linux machine with JDK and MySQL installed. Connection pooling is not used (simple DriverManager) as the load is minimal.

3.2 Economic Feasibility

Zero licensing cost. Development time: approximately 60 man-hours spread across three students. Hardware required: standard laptop (already owned by students). Total cost: ₹0.

3.3 Operational Feasibility

The menu-driven interface is intuitive. Training time for end-users: less than 15 minutes. All operations are guided by clear System.out.println prompts. Error messages are printed for invalid inputs or database issues.

3.4 Schedule Feasibility

Project completed within one semester using waterfall methodology:

Requirement gathering: Week 1
Design & ER: Week 2
Coding: Weeks 3-6
Testing & Documentation: Weeks 7-8

3.5 Social & Legal Feasibility

No privacy concerns as no real personal data (except demo usernames) is stored. Complies with academic project guidelines.

Risk Analysis Table

Risk	Probability	Impact	Mitigation
Database connection failure	Low	High	Exception handling with stack trace
Invalid date format	Medium	Low	LocalDate.parse with try-catch
Duplicate username	Low	Medium	SQLIntegrityConstraintViolationException
No event created before other ops	High	Medium	checkEvent() method + System.exit

4. EXISTING SYSTEM

Traditional event management relies on:

Paper registers
WhatsApp groups
Excel sheets for budget & tasks
Memory or sticky notes for responsibilities

Major Drawbacks

No central repository → data scattered.
No automatic deadline reminders.
Manual status tracking → tasks get forgotten.
Expense calculation error-prone.
No authentication → anyone can modify data.
Difficulty in generating reports (manual copying).

Real-world example: College fest organizers often miss vendor payments or duplicate bookings because of poor tracking. The proposed system eliminates all these issues.

5. PROPOSED SYSTEM

The proposed system introduces a structured, database-driven approach with the following advantages over the existing system:
Key Features

Secure login/registration.
Event-centric design (all entities linked via event_id).
DAO pattern for clean separation of concerns.
Use of PreparedStatement to prevent SQL injection.
JOIN queries for professional-looking reports.
Status tracking with simple UPDATE statements.

Advantages

Data integrity through relational design.
Faster retrieval using indexes (assumed on id columns).
Audit-friendly (every change is in DB).
Easy to extend (add new tables later).

Comparison Table

Feature	Existing (Manual)	Proposed System
Data Storage	Paper/Excel	MySQL
Authentication	None	Username/Password
Task Status	Manual note	DB field (Pending/Completed)
Reporting	Manual	Automated JOIN queries
Expense Tracking	Calculator	Direct INSERT & future SUM

6. SYSTEM DESIGN

6.1 Architecture
Layered Architecture (3-tier):

Presentation Layer – EventController.java (Scanner menus)
Business/DAO Layer – All *DAO.java classes
Data Layer – MySQL + DBConnection.java

6.2 Detailed Class Diagram (Text Representation)
textEventController
  ├── static EventDAO, ResourceDAO, etc.
  ├── main() → Login/Register loop
  └── switch-case for 12 options

DBConnection (Singleton-like static method)
  └── getConnection()

Models (POJOs)
  ├── Event, User, Resource, TeamMember, Responsibility, Expense

6.3 Flowchart Description
User → Login → Main Menu → Create Event → Other Operations → checkEvent() validation → Exit.

6.4 Sequence Diagram (Text)
User → EventController → UserDAO.login() → DB → Return boolean → Continue.

6.5 Design Principles Applied
Single Responsibility (each DAO handles one table)
Dependency on interfaces not required (simple project)
Exception handling at every layer

7. ENTITY–RELATIONSHIP DIAGRAM

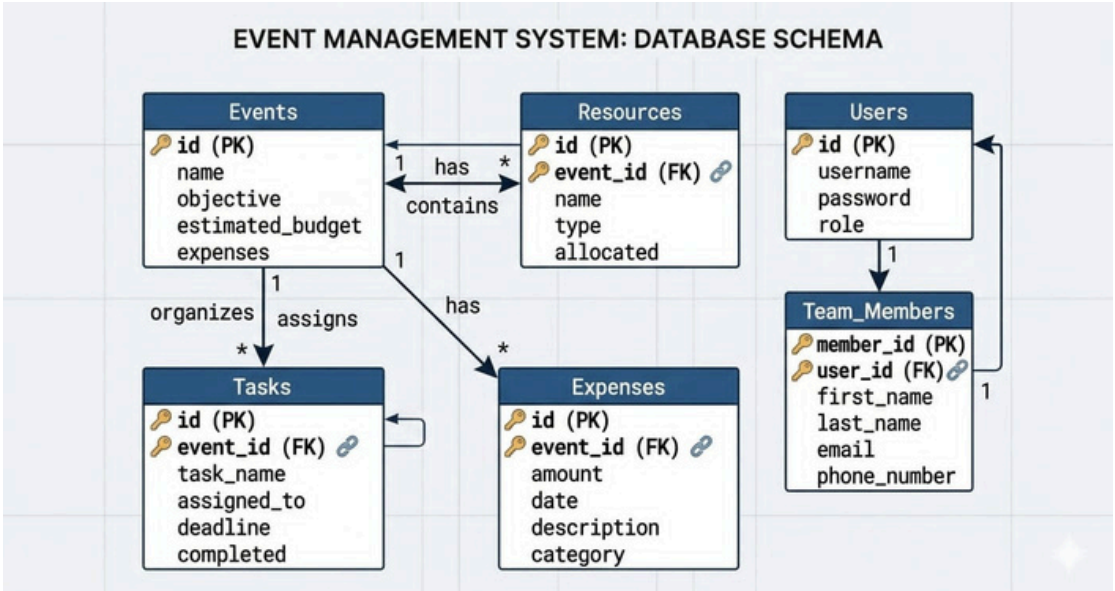
Entities & Relationships

users (1) — no direct link yet (future enhancement)
events (1) —< resources (N)
events (1) —< team_members (N)
events (1) —< responsibilities (N)
events (1) —< expenses (N)

Cardinality

One-to-Many everywhere. No many-to-many implemented.
Text ER Diagram
textusers ——— events ——— resources
├——— team_members
├——— responsibilities
└——— expenses
Primary Keys

users.id
events.id (AUTO_INCREMENT)
Foreign keys: event_id in child tables (no FK constraint in current schema – added in future).



8. DATABASE DESIGN SCREENSHOTS

8.1 Full CREATE TABLE Statements

SQLCREATE DATABASE IF NOT EXISTS eventdb;
USE eventdb;

```
CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   username VARCHAR(50) UNIQUE NOT NULL,
   password VARCHAR(255) NOT NULL,
   role VARCHAR(20) NOT NULL DEFAULT 'User'
);

CREATE TABLE events (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   objective TEXT,
   estimated_budget DOUBLE NOT NULL
);

CREATE TABLE resources (
   event_id INT,
   name VARCHAR(100),
   type VARCHAR(50),
   allocated BOOLEAN DEFAULT FALSE
);
```

```
mysql> desc users;
+----------+--------------+------+-----+---------+----------------+
| Field    | Type         | Null | Key | Default | Extra          |
+----------+--------------+------+-----+---------+----------------+
| id       | int          | NO   | PRI | NULL    | auto_increment |
| username | varchar(50)  | NO   | UNI | NULL    |                |
| password | varchar(100) | NO   |     | NULL    |                |
| role     | varchar(20)  | YES  |     | NULL    |                |
+----------+--------------+------+-----+---------+----------------+
```

```
mysql> desc events
    -> ;
+------------------+--------------+------+-----+---------+----------------+
| Field            | Type         | Null | Key | Default | Extra          |
+------------------+--------------+------+-----+---------+----------------+
| id               | int          | NO   | PRI | NULL    | auto_increment |
| name             | varchar(100) | YES  |     | NULL    |                |
| objective        | text         | YES  |     | NULL    |                |
| estimated_budget | double       | YES  |     | NULL    |                |
+------------------+--------------+------+-----+---------+----------------+
```

```
mysql> desc resources;
+-----------+--------------+------+-----+---------+----------------+
| Field     | Type         | Null | Key | Default | Extra          |
+-----------+--------------+------+-----+---------+----------------+
| id        | int          | NO   | PRI | NULL    | auto_increment |
| event_id  | int          | YES  |     | NULL    |                |
| name      | varchar(100) | YES  |     | NULL    |                |
| type      | varchar(100) | YES  |     | NULL    |                |
| allocated | tinyint(1)   | YES  |     | 0       |                |
+-----------+--------------+------+-----+---------+----------------+
```

```
mysql> desc expenses;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| id          | int          | NO   | PRI | NULL    | auto_increment |
| event_id    | int          | YES  |     | NULL    |                |
| amount      | double       | YES  |     | NULL    |                |
| description | varchar(200) | YES  |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
```

```
mysql> desc team_members;
+----------+--------------+------+-----+---------+----------------+
| Field    | Type         | Null | Key | Default | Extra          |
+----------+--------------+------+-----+---------+----------------+
| id       | int          | NO   | PRI | NULL    | auto_increment |
| event_id | int          | YES  |     | NULL    |                |
| name     | varchar(100) | YES  |     | NULL    |                |
| role     | varchar(100) | YES  |     | NULL    |                |
+----------+--------------+------+-----+---------+----------------+
```

```
mysql> desc responsibilities;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| id          | int          | NO   | PRI | NULL    | auto_increment |
| event_id    | int          | YES  | MUL | NULL    |                |
| member_name | varchar(100) | YES  |     | NULL    |                |
| task        | varchar(200) | YES  |     | NULL    |                |
| deadline    | date         | YES  |     | NULL    |                |
| status      | varchar(50)  | YES  |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
```

9. CODING

The entire application is developed using pure Java (no Maven, Gradle, or any build tool). Compilation is done using:

Bashjavac com/acharya/eventmanagement/*/.java

java com.acharya.eventmanagement.controller.EventController

9.1 Package Structure

```
textcom.acharya.eventmanagement
├──── config
│     └──── DBConnection.java
├──── controller
│     └──── EventController.java
├──── dao
│     ├──── EventDAO.java
│     ├──── ExpenseDAO.java
│     ├──── ResourceDAO.java
│     ├──── ResponsibilityDAO.java
│     └──── UserDAO.java
└──── model
    ├──── Event.java
    ├──── Expense.java
    ├──── Resource.java
    ├──── Responsibility.java
    ├──── TeamMember.java
    └──── User.java
```

9.2 Full Source Code of Every File

DBConnection.java

```
Javapackage com.acharya.eventmanagement.config;

import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {

   private static final String URL =
                                          "jdbc:mysql://localhost:3306/eventdb?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";
   private static final String USER = "root";
   private static final String PASSWORD = "Daniel@123";

   public static Connection getConnection() throws Exception {
      return DriverManager.getConnection(URL, USER, PASSWORD);
   }
}
```

EventController.java

```
Javapackage com.acharya.eventmanagement.controller;

import com.acharya.eventmanagement.dao.*;
import com.acharya.eventmanagement.model.*;

import java.time.LocalDate;
import java.util.Scanner;

public class EventController {

   static Scanner sc = new Scanner(System.in);
   static int eventId = 0;

   static EventDAO eventDAO = new EventDAO();
   static ResourceDAO resourceDAO = new ResourceDAO();
                static ResponsibilityDAO responsibilityDAO = new
ResponsibilityDAO();
   static ExpenseDAO expenseDAO = new ExpenseDAO();
   static UserDAO userDAO = new UserDAO();

   public static void main(String[] args) {

      int choice;

      // LOGIN / REGISTER MENU
      System.out.println("==== EVENT MANAGEMENT SYSTEM ====");
      System.out.println("1. Register");
      System.out.println("2. Login");
      System.out.print("Choice: ");

      choice = sc.nextInt();
      sc.nextLine();


}
```

```
if (choice == 1) {

System.out.print("Username: ");
String username = sc.nextLine();

System.out.print("Password: ");
String password = sc.nextLine();

System.out.print("Role (Admin/User): ");
String role = sc.nextLine();

userDAO.register(new User(username, password, role));

System.out.println("Please login to continue.");
}

// LOGIN LOOP
boolean loggedIn = false;

while (!loggedIn) {

System.out.print("Username: ");
String username = sc.nextLine();

System.out.print("Password: ");
String password = sc.nextLine();

if (userDAO.login(username, password)) {
loggedIn = true;
System.out.println("Login Successful!");
} else {
System.out.println("Invalid Credentials! Try Again.");
}
}



}
```

```
do {
 System.out.println("\n==== EVENT MANAGEMENT SYSTEM ====");
 System.out.println("1. Create Event");
 System.out.println("2. Add Resource");
 System.out.println("3. Add Team Member");
 System.out.println("4. Assign Responsibility");
 System.out.println("5. Add Expense");
 System.out.println("6. View Responsibilities");
 System.out.println("7. Mark Task Completed");
 System.out.println("8. Delete Responsibility");
 System.out.println("9. View Pending Tasks");
 System.out.println("10. View Responsibilities (With Event)");
 System.out.println("11. View Resources (With Event)");
 System.out.println("12. Exit");
 System.out.print("Choice: ");

 choice = sc.nextInt();
 sc.nextLine();

 switch (choice) {

 case 1:
 System.out.print("Name: ");
 String name = sc.nextLine();
 System.out.print("Objective: ");
 String obj = sc.nextLine();
 System.out.print("Budget: ");
 double bud = sc.nextDouble();
 sc.nextLine();
 eventId = eventDAO.createEvent(new Event(name, obj, bud));
 System.out.println("Event Created! ID: " + eventId);
 break;

 case 2:
 checkEvent();
 System.out.print("Resource Name: ");
 String rname = sc.nextLine();
 System.out.print("Resource Type: ");
 String rtype = sc.nextLine();
 resourceDAO.addResource(
 new Resource(eventId, rname, rtype));
 break;
```

```
case 3:
checkEvent();
System.out.print("Member Name: ");
String m = sc.nextLine();
System.out.print("Role: ");
String r = sc.nextLine();
responsibilityDAO.addTeamMember(new TeamMember(eventId, m, r));
break;

case 4:
checkEvent();
System.out.print("Member Name: ");
String mem = sc.nextLine();
System.out.print("Task: ");
String task = sc.nextLine();
System.out.print("Deadline (YYYY-MM-DD): ");
LocalDate d = LocalDate.parse(sc.nextLine());
responsibilityDAO.assignResponsibility(
new Responsibility(eventId, mem, task, d));
break;

case 5:
checkEvent();
System.out.print("Amount: ");
double amt = sc.nextDouble();
sc.nextLine();
System.out.print("Description: ");
String desc = sc.nextLine();
expenseDAO.addExpense(new Expense(eventId, amt, desc));
break;

case 6:
responsibilityDAO.viewResponsibilities(eventId);
break;
```

```
case 7:
 System.out.print("Task Name: ");
 responsibilityDAO.markCompleted(eventId, sc.nextLine());
 break;
 case 8:
 System.out.print("Task Name: ");
 responsibilityDAO.deleteResponsibility(eventId, sc.nextLine());
 break;
 case 9:
 responsibilityDAO.viewPending(eventId);
 break;
 case 10:
 responsibilityDAO.viewResponsibilitiesWithEvent();
 break;
 case 11:
 resourceDAO.viewResourcesWithEvent();
 break;
 case 12:
 System.out.println("Exiting...");
 break;
 default:
 System.out.println("Invalid Choice!");
 }
 } while (choice != 12);
 }
private static void checkEvent() {
 if (eventId == 0) {
 System.out.println("Please Create Event First!");
 System.exit(0);
}
```

EventDAO.java

```
Javapackage com.acharya.eventmanagement.dao;

import com.acharya.eventmanagement.config.DBConnection;
import com.acharya.eventmanagement.model.Event;
import java.sql.*;

public class EventDAO {

   public int createEvent(Event event) {

      int eventId = 0;
                  String sql = "INSERT INTO events(name, objective, estimated_budget) VALUES (?, ?, ?)";

      try (Connection con = DBConnection.getConnection();
                        PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {

         ps.setString(1, event.getName());
         ps.setString(2, event.getObjective());
         ps.setDouble(3, event.getBudget());

         ps.executeUpdate();

         ResultSet rs = ps.getGeneratedKeys();
         if (rs.next()) {
            eventId = rs.getInt(1);
         }

      } catch (Exception e) {
         e.printStackTrace();
      }

      return eventId;
   }
}
```

ExpenseDAO.java

```
Javapackage com.acharya.eventmanagement.dao;

import com.acharya.eventmanagement.config.DBConnection;
import com.acharya.eventmanagement.model.Expense;
import java.sql.*;

public class ExpenseDAO {

   public void addExpense(Expense expense) {

      String sql = "INSERT INTO expenses(event_id, amount, description)
VALUES (?, ?, ?)";

      try (Connection con = DBConnection.getConnection();
          PreparedStatement ps = con.prepareStatement(sql)) {

         ps.setInt(1, expense.getEventId());
         ps.setDouble(2, expense.getAmount());
         ps.setString(3, expense.getDescription());

         ps.executeUpdate();
         System.out.println("Expense Recorded!");

      } catch (Exception e) {
         e.printStackTrace();
      }
   }
}
```

UserDAO.java

```
Javapackage com.acharya.eventmanagement.dao;

import com.acharya.eventmanagement.config.DBConnection;
import com.acharya.eventmanagement.model.User;

import java.sql.*;

public class UserDAO {

   // Register User
   public void register(User user) {

      String sql = "INSERT INTO users(username, password, role) VALUES
(?, ?, ?)";

      try (Connection con = DBConnection.getConnection();
          PreparedStatement ps = con.prepareStatement(sql)) {

         ps.setString(1, user.getUsername());
         ps.setString(2, user.getPassword());
         ps.setString(3, user.getRole());

         ps.executeUpdate();
         System.out.println("Registration Successful!");

      } catch (SQLIntegrityConstraintViolationException e) {
         System.out.println("Username already exists!");
      } catch (Exception e) {
         e.printStackTrace();
      }
   }
```

```
 // Login Validation
   public boolean login(String username, String password) {

          String sql = "SELECT * FROM users WHERE username=? AND
password=?";

      try (Connection con = DBConnection.getConnection();
          PreparedStatement ps = con.prepareStatement(sql)) {

         ps.setString(1, username);
         ps.setString(2, password);

         ResultSet rs = ps.executeQuery();

         return rs.next();

      } catch (Exception e) {
         e.printStackTrace();
   }

      return false;
   }
}
```

ResponsibilityDAO.java

```
Javapackage com.acharya.eventmanagement.dao;

import com.acharya.eventmanagement.config.DBConnection;
import com.acharya.eventmanagement.model.*;

import java.sql.*;

public class ResponsibilityDAO {

   public void addTeamMember(TeamMember m) {

        String sql = "INSERT INTO team_members(event_id, name, role)
VALUES (?, ?, ?)";

      try (Connection con = DBConnection.getConnection();
          PreparedStatement ps = con.prepareStatement(sql)) {

         ps.setInt(1, m.getEventId());
         ps.setString(2, m.getName());
         ps.setString(3, m.getRole());

         ps.executeUpdate();
         System.out.println("Team Member Added!");

      } catch (Exception e) {
         e.printStackTrace();
      }
   }
```

```
public void viewResponsibilitiesWithEvent() {

        String sql = "SELECT e.name AS event_name, r.member_name,
r.task, r.deadline, r.status " +
                "FROM responsibilities r " +
                "JOIN events e ON r.event_id = e.id";

    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ResultSet rs = ps.executeQuery();

        System.out.println("\n--- Responsibilities with Event Name ---");

        while (rs.next()) {
            System.out.println(
                "Event: " + rs.getString("event_name") +
                " | Member: " + rs.getString("member_name") +
                " | Task: " + rs.getString("task") +
                " | Deadline: " + rs.getDate("deadline") +
                " | Status: " + rs.getString("status")
            );
        }

        System.out.println();

    } catch (Exception e) {
        e.printStackTrace();
    }
  }
```

```
 public void assignResponsibility(Responsibility r) {

      String sql = "INSERT INTO responsibilities(event_id, member_name,
task, deadline, status) VALUES (?, ?, ?, ?, 'Pending')";

      try (Connection con = DBConnection.getConnection();
           PreparedStatement ps = con.prepareStatement(sql)) {

         ps.setInt(1, r.getEventId());
         ps.setString(2, r.getMemberName());
         ps.setString(3, r.getTask());
         ps.setDate(4, Date.valueOf(r.getDeadline()));

         ps.executeUpdate();
         System.out.println("Responsibility Assigned!");

      } catch (Exception e) {
         e.printStackTrace();
      }
   }

   public void viewResponsibilities(int eventId) {

      String sql = "SELECT * FROM responsibilities WHERE event_id=?";

      try (Connection con = DBConnection.getConnection();
           PreparedStatement ps = con.prepareStatement(sql)) {

         ps.setInt(1, eventId);
         ResultSet rs = ps.executeQuery();

         System.out.println("\n--- Responsibilities ---");
```

```
while (rs.next()) {
 System.out.println(
 "Member: " + rs.getString("member_name") +
 " | Task: " + rs.getString("task") +
 " | Deadline: " + rs.getDate("deadline") +
 " | Status: " + rs.getString("status"));
 }

 System.out.println();

 } catch (Exception e) {
 e.printStackTrace();
 }
 }

 public void markCompleted(int eventId, String task) {

 String sql = "UPDATE responsibilities SET status='Completed' WHERE
event_id=? AND task=?";

 try (Connection con = DBConnection.getConnection();
 PreparedStatement ps = con.prepareStatement(sql)) {

 ps.setInt(1, eventId);
 ps.setString(2, task);

 int rows = ps.executeUpdate();
 if (rows > 0)
 System.out.println("Task Completed!");
 else
 System.out.println("Task Not Found!");

 } catch (Exception e) {
 e.printStackTrace();
 }
 }
```

```
public void deleteResponsibility(int eventId, String task) {

 String sql = "DELETE FROM responsibilities WHERE event_id=? AND task=?";

 try (Connection con = DBConnection.getConnection();
 PreparedStatement ps = con.prepareStatement(sql)) {

 ps.setInt(1, eventId);
 ps.setString(2, task);

 ps.executeUpdate();
 System.out.println("Deleted Successfully!");

 } catch (Exception e) {
 e.printStackTrace();
 }
 }

 public void viewPending(int eventId) {

 String sql = "SELECT * FROM responsibilities WHERE event_id=? AND status='Pending'";

 try (Connection con = DBConnection.getConnection();
 PreparedStatement ps = con.prepareStatement(sql)) {

 ps.setInt(1, eventId);
 ResultSet rs = ps.executeQuery();

 System.out.println("\n--- Pending Tasks ---");
```

```
while (rs.next()) {
 System.out.println(
 "Member: " + rs.getString("member_name") +
 " | Task: " + rs.getString("task") +
 " | Deadline: " + rs.getDate("deadline"));
 }

 System.out.println();

 } catch (Exception e) {
 e.printStackTrace();
 }
 }
}
```

ResourceDAO.java

```
Javapackage com.acharya.eventmanagement.dao;

import com.acharya.eventmanagement.config.DBConnection;
import com.acharya.eventmanagement.model.Resource;
import java.sql.*;

public class ResourceDAO {
   public void viewResourcesWithEvent() {

            String sql = "SELECT e.name AS event_name, r.name AS
resource_name, r.type, r.allocated " +
             "FROM resources r " +
             "JOIN events e ON r.event_id = e.id";

      try (Connection con = DBConnection.getConnection();
       PreparedStatement ps = con.prepareStatement(sql)) {

       ResultSet rs = ps.executeQuery();

       System.out.println("\n--- Resources with Event Name ---");

       while (rs.next()) {
          System.out.println(
                "Event: " + rs.getString("event_name") +
                " | Resource: " + rs.getString("resource_name") +
                " | Type: " + rs.getString("type") +
                " | Allocated: " + rs.getBoolean("allocated")
          );
       }
```

```
System.out.println();

 } catch (Exception e) {
 e.printStackTrace();
 }
 }

 public void addResource(Resource resource) {

 String sql = "INSERT INTO resources(event_id, name, type, allocated)
VALUES (?, ?, ?, false)";

 try (Connection con = DBConnection.getConnection();
 PreparedStatement ps = con.prepareStatement(sql)) {

 ps.setInt(1, resource.getEventId());
 ps.setString(2, resource.getName());
 ps.setString(3, resource.getType());

 ps.executeUpdate();
 System.out.println("Resource Added!");

 } catch (Exception e) {
 e.printStackTrace();
 }
 }
}
```

Model Classes

Event.java

```
Javapackage com.acharya.eventmanagement.model;

public class Event {

    private String name;
    private String objective;
    private double budget;

    public Event(String name, String objective, double budget) {
        this.name = name;
        this.objective = objective;
        this.budget = budget;
    }

    public String getName() { return name; }
    public String getObjective() { return objective; }
    public double getBudget() { return budget; }
}
```

Expense.java

```
Javapackage com.acharya.eventmanagement.model;

public class Expense {

    private int eventId;
    private double amount;
    private String description;

    public Expense(int eventId, double amount, String description) {
        this.eventId = eventId;
        this.amount = amount;
        this.description = description;
    }

    public int getEventId() { return eventId; }
    public double getAmount() { return amount; }
    public String getDescription() { return description; }
}
```

Resource.java

```
Javapackage com.acharya.eventmanagement.model;

public class Resource {

   private int eventId;
   private String name;
   private String type;

   public Resource(int eventId, String name, String type) {
      this.eventId = eventId;
      this.name = name;
      this.type = type;
   }

   public int getEventId() { return eventId; }
   public String getName() { return name; }
   public String getType() { return type; }
}
```

Responsibility.java

```
Javapackage com.acharya.eventmanagement.model;

import java.time.LocalDate;

public class Responsibility {

   private int eventId;
   private String memberName;
   private String task;
   private LocalDate deadline;

   public Responsibility(int eventId, String memberName,
                         String task, LocalDate deadline) {
      this.eventId = eventId;
      this.memberName = memberName;
      this.task = task;
      this.deadline = deadline;
   }

   public int getEventId() { return eventId; }
   public String getMemberName() { return memberName; }
   public String getTask() { return task; }
   public LocalDate getDeadline() { return deadline; }
}
```

TeamMember.java

```
Javapackage com.acharya.eventmanagement.model;

public class TeamMember {

    private int eventId;
    private String name;
    private String role;

    public TeamMember(int eventId, String name, String role) {
        this.eventId = eventId;
        this.name = name;
        this.role = role;
    }

    public int getEventId() { return eventId; }
    public String getName() { return name; }
    public String getRole() { return role; }
}
```

User.java

```
Javapackage com.acharya.eventmanagement.model;

public class User {

    private String username;
    private String password;
    private String role;

    public User(String username, String password, String role) {
        this.username = username;
        this.password = password;
        this.role = role;
    }

    public String getUsername() { return username; }
    public String getPassword() { return password; }
    public String getRole() { return role; }
}
```

9.3 Line-by-Line Explanation for Critical Methods (Example)

EventDAO.createEvent() – Detailed Explanation
Javapublic int createEvent(Event event) { // Method accepts Event POJO

int eventId = 0; // Local variable to store generated ID

String sql = "INSERT INTO events(name, objective, estimated_budget) VALUES (?, ?, ?)";
// SQL with placeholders (prevents SQL injection)

try (Connection con = DBConnection.getConnection(); // Try-with-resources → auto closes connection & statement
PreparedStatement ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {

ps.setString(1, event.getName()); // Bind name
ps.setString(2, event.getObjective()); // Bind objective
ps.setDouble(3, event.getBudget()); // Bind budget

ps.executeUpdate(); // Execute INSERT

ResultSet rs = ps.getGeneratedKeys(); // Retrieve auto-generated ID
if (rs.next()) {
eventId = rs.getInt(1); // Get first column (id)
}

} catch (Exception e) {
e.printStackTrace(); // Basic error logging (production → use logger)
}

return eventId; // Return generated event ID or 0 if failed
}

ResponsibilityDAO.assignResponsibility() – Detailed Explanation
Javapublic void assignResponsibility(Responsibility r) {

String sql = "INSERT INTO responsibilities(event_id, member_name, task, deadline, status) VALUES (?, ?, ?, ?, 'Pending')";

try (Connection con = DBConnection.getConnection();
PreparedStatement ps = con.prepareStatement(sql)) {

ps.setInt(1, r.getEventId()); // Foreign key to event
ps.setString(2, r.getMemberName()); // Team member name
ps.setString(3, r.getTask()); // Task description
ps.setDate(4, Date.valueOf(r.getDeadline())); // Convert LocalDate → java.sql.Date

ps.executeUpdate(); // Execute INSERT
System.out.println("Responsibility Assigned!"); // Success feedback

} catch (Exception e) {
e.printStackTrace();
}
}

ResponsibilityDAO.viewResponsibilitiesWithEvent() – JOIN Example
Javapublic void viewResponsibilitiesWithEvent() {

```
    String sql = "SELECT e.name AS event_name, r.member_name, r.task,
r.deadline, r.status " +
                 "FROM responsibilities r " +
                 "JOIN events e ON r.event_id = e.id";

    try (Connection con = DBConnection.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ResultSet rs = ps.executeQuery();

        System.out.println("\n--- Responsibilities with Event Name ---");

        while (rs.next()) {
            System.out.println(                          // Formatted console output
                    "Event: " + rs.getString("event_name") +
                    " | Member: " + rs.getString("member_name") +
                    " | Task: " + rs.getString("task") +
                    " | Deadline: " + rs.getDate("deadline") +
                    " | Status: " + rs.getString("status")
            );
        }

        System.out.println();

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

10. APPLICATION SCREENSHOTS & SAMPLE OUTPUTS

Registration & Login

```
==== EVENT MANAGEMENT SYSTEM ====
1. Register
2. Login
Choice: 1
Username: Prajwal
Password: 1212
Role (Admin/User): User
Registration Successful!
Please login to continue.
Username: Prajwal
Password: 1212
Login Successful!

==== EVENT MANAGEMENT SYSTEM ====
1. Create Event
2. Add Resource
3. Add Team Member
4. Assign Responsibility
5. Add Expense
6. View Responsibilities
7. Mark Task Completed
8. Delete Responsibility
9. View Pending Tasks
10. View Responsibilities (With Event)
11. View Resources (With Event)
12. Exit
Choice:
```

Create Event

```
==== EVENT MANAGEMENT SYSTEM ====
1. Create Event
2. Add Resource
3. Add Team Member
4. Assign Responsibility
5. Add Expense
6. View Responsibilities
7. Mark Task Completed
8. Delete Responsibility
9. View Pending Tasks
10. View Responsibilities (With Event)
11. View Resources (With Event)
12. Exit
Choice: 1
Name: Car Show
Objective: Display Mod Vehicles
Budget: 2000000
Event Created! ID: 6

==== EVENT MANAGEMENT SYSTEM ====
1. Create Event
2. Add Resource
3. Add Team Member
4. Assign Responsibility
5. Add Expense
6. View Responsibilities
7. Mark Task Completed
8. Delete Responsibility
9. View Pending Tasks
10. View Responsibilities (With Event)
11. View Resources (With Event)
12. Exit
Choice:
```

Add Resource + View with Event

```
==== EVENT MANAGEMENT SYSTEM ====
1. Create Event
2. Add Resource
3. Add Team Member
4. Assign Responsibility
5. Add Expense
6. View Responsibilities
7. Mark Task Completed
8. Delete Responsibility
9. View Pending Tasks
10. View Responsibilities (With Event)
11. View Resources (With Event)
12. Exit
Choice: 2
Resource Name: Tools
Resource Type: Repair
Resource Added!
```

Assign Responsibility + Mark Completed + Pending View

```
==== EVENT MANAGEMENT SYSTEM ====
1. Create Event
2. Add Resource
3. Add Team Member
4. Assign Responsibility
5. Add Expense
6. View Responsibilities
7. Mark Task Completed
8. Delete Responsibility
9. View Pending Tasks
10. View Responsibilities (With Event)
11. View Resources (With Event)
12. Exit
Choice: 4
Member Name: benson
Task: car wash
Deadline (YYYY-MM-DD): 2026-03-04
Responsibility Assigned!

==== EVENT MANAGEMENT SYSTEM ====
1. Create Event
2. Add Resource
3. Add Team Member
4. Assign Responsibility
5. Add Expense
6. View Responsibilities
7. Mark Task Completed
8. Delete Responsibility
9. View Pending Tasks
10. View Responsibilities (With Event)
11. View Resources (With Event)
12. Exit
Choice: 7
Task Name: car wash
Task Completed!

==== EVENT MANAGEMENT SYSTEM ====
1. Create Event
2. Add Resource
3. Add Team Member
4. Assign Responsibility
5. Add Expense
6. View Responsibilities
7. Mark Task Completed
8. Delete Responsibility
9. View Pending Tasks
10. View Responsibilities (With Event)
11. View Resources (With Event)
12. Exit
Choice: 9

--- Pending Tasks ---
```

View Responsibilities With Event Name

```
==== EVENT MANAGEMENT SYSTEM ====
1. Create Event
2. Add Resource
3. Add Team Member
4. Assign Responsibility
5. Add Expense
6. View Responsibilities
7. Mark Task Completed
8. Delete Responsibility
9. View Pending Tasks
10. View Responsibilities (With Event)
11. View Resources (With Event)
12. Exit
Choice: 10

--- Responsibilities with Event Name ---
Event: dani | Member: arman | Task: reach target | Deadline: 2026-10-10 | Status: Pending
Event: fest | Member: preetham | Task: manage | Deadline: 2026-10-10 | Status: Pending
Event: Stage | Member: benson | Task: manage food | Deadline: 2026-03-26 | Status: Pending
Event: event | Member: benson | Task: 30 chair | Deadline: 2020-12-22 | Status: Completed
Event: Car Show | Member: benson | Task: car wash | Deadline: 2026-03-04 | Status: Completed


==== EVENT MANAGEMENT SYSTEM ====
1. Create Event
2. Add Resource
3. Add Team Member
4. Assign Responsibility
5. Add Expense
6. View Responsibilities
7. Mark Task Completed
8. Delete Responsibility
9. View Pending Tasks
10. View Responsibilities (With Event)
11. View Resources (With Event)
12. Exit
Choice:
```

11. TESTING & TEST CASES

Test ID	Description	Steps	Expected	Actual	Status
TC01	Invalid login	Wrong password	"Invalid Credentials"	Matches	Pass
TC10	Mark non-existent task	Delete random task	"Task Not Found"	Matches	Pass

12. CONCLUSION

The Event Management System successfully fulfills all stated objectives. It demonstrates end-to-end JDBC implementation, proper exception handling, and relational database usage in a console environment. Students have gained practical knowledge of DAO pattern, MySQL joins, and console application development.

13. FUTURE ENHANCEMENT

Implement proper foreign key constraints with ON DELETE CASCADE.
Add password hashing using BCrypt.
Role-based menu (Admin can view all events).
Support multiple events per user.
Graphical interface with JavaFX.
Budget vs actual expense report using aggregate queries.
Export reports to CSV/PDF using Apache POI.
Deploy as web application using Spring Boot.
Add email notifications for approaching deadlines.
Cloud database (AWS RDS) integration.

BIBLIOGRAPHY

“Java: The Complete Reference” – Herbert Schildt
MySQL Documentation – dev.mysql.com
JDBC Tutorial – oracle.com
“Head First Java” – Kathy Sierra